Optimizers, hyperparameter transfer, and annealed comparisons

> **Reading note.** This is an explanatory reconstruction of Jiaxuan Zou's literature synthesis. Published experiments belong to the cited teams. Numbered protocols and illustrative matrices are our planning synthesis unless explicitly attributed.

The blog's §6 distinguishes a tuned frontier from a fixed-recipe ladder. Comparing a newly tuned optimizer to a default baseline confounds the algorithm with tuning effort. Comparing stable-phase loss can also select a different winner from comparing fully annealed endpoints.

Tune and transfer

1. Freeze optimizer equations, parameter groups, initialization, parameterization, clipping, precision, and reduction conventions.
2. At a reference (N, D), search peak learning rate η, token batch size B, and weight decay λ with a comparable budget for every candidate.
3. Repeat at another size and horizon to estimate transfer trends rather than copying one optimum.
4. Fit a candidate relation such as $\eta_* = aN^pD^qB^r$ in a declared parameterization. This is a hypothesis to test; do not treat exponents from another recipe as constants of nature.
5. Recheck configurations around the predicted optimum at an intermediate scale. Extend the search if the best result lies at a boundary.
6. Compare complete schedules at their annealed endpoints, with fresh-seed confirmation of finalists.
7. Run stability stress tests and a held-out scale before committing to the target.

For intuition, a constant-learning-rate AdamW decay-only update multiplies weights by $(1 - \eta\lambda)$ each step. Over $S = D/B$ steps,

$$(1 - \eta\lambda)^S \approx e^{-\eta\lambda D/B}.$$

Thus the dimensionless decay-timescale proxy $\tau = B/(\eta\lambda D)$ couples batch size, tokens, learning rate, and weight decay. This derivation assumes constant $\eta\lambda$ and isolates decay from gradient updates. For a schedule, use the actual sum of learning rates; it does not prove that a fixed τ is universally optimal.

Concrete fits and their conditions

Power Lines v2 finds $\tau_* \approx 1.084(D/N)^{-0.527}$, rather than a constant optimum across TPP. Its batch law $B_* = 0.0306D^{0.383}$ is reported in **sequences of 2,048 tokens**. At $D = 10^{11}$ tokens, its table projects approximately 500 sequences, or 1,024,000 tokens per batch. This is a fitted projection under its tuned-decay AdamW/µP recipe. Bergsma et al., v2, Fig. 1/Table 5.

Step Law v3 instead reports $B_* = 0.58D^{0.571}$ in tokens under its joint LR/batch tuning procedure. Its raw LR coefficients must not be applied automatically as µP base-LR coefficients. Li et al., v3, Table 1.

The architecture evidence guide gives the full AI2 LR/batch table, a matrix-specific µP transfer example, and the limitations of depth transfer.

Minimal sweep and decision record

Stage	Sweep	Readout
Reference	LR × batch, then decay refinement	Annealed loss and stability
Transfer	Sizes × token horizons	Optimum region and transfer residual
Finalists	Independent seed repeats	Paired improvement and uncertainty
Holdout	Frozen transferred configuration	Prediction and stability error

Maintain schedule-specific fits when comparing cosine and WSD. Under μP, follow the parameterization's matrix-specific learning-rate and initialization rules; an ordinary width change is not automatically a valid μP transfer.

Acceptance checklist: comparable tuning; endpoint evaluation; fresh confirmation seeds; measured time per update; no unexplained gradient growth; intact optimizer state after resume; held-out target-direction test.

Sources: blog §6; Fantastic Optimizers, Wen et al., 2025; Step Law, 2025; Power Lines, 2025; Tensor Programs V, 2022.

Blog/notes crosswalk · Detailed companion